

CSE 3203: Design and Analysis of Algorithms – II

Supplementary Proofs

Part A: 3-SAT and Clique are Equally Hard

Definitions

3-SAT (decision). Given a boolean formula $\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$ in conjunctive normal form, where every clause C_i has exactly 3 literals, decide whether ϕ is satisfiable.

Clique (decision). Given a graph $G = (V, E)$ and an integer k , decide whether G contains a *clique* of size k (i.e., k vertices that are pairwise adjacent).

”Equally hard” means the two problems are polynomial-time *interreducible*: $3\text{-SAT} \leq_p \text{Clique}$ and $\text{Clique} \leq_p 3\text{-SAT}$. We already know (by the Cook–Levin theorem) that 3-SAT is NP-complete. We prove both directions below; together with the fact that both problems are NP-complete, this establishes that they are equally hard.

Step 1: $3\text{-SAT} \leq_p \text{Clique}$

Let $\phi = C_1 \wedge \cdots \wedge C_m$ be a 3-CNF formula, with $C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3})$.

Construction. Build a graph $G = (V, E)$:

- Create one vertex v_{ja} for every literal occurrence ℓ_{ja} , $1 \leq j \leq m$, $1 \leq a \leq 3$ (so $|V| = 3m$). Group the 3 vertices belonging to the same clause C_j together; add **no** edges within a group.
- For two vertices v_{ja} and v_{kb} belonging to *different* clauses ($j \neq k$), add an edge between them **if and only if** the corresponding literals are *not* complementary, i.e. it is *not* the case that $\ell_{ja} = x$ and $\ell_{kb} = \neg x$ for the same variable x (or vice-versa).

Set $k = m$.

This construction has $O(m)$ vertices and $O(m^2)$ edges and is clearly computable in polynomial time.

Correctness.

- $(\Rightarrow) \phi \text{ satisfiable} \Rightarrow G \text{ has a clique of size } m$. Fix a satisfying assignment. Each clause has at least one true literal; choose one true literal per clause and take its vertex, giving m vertices $v_{1a_1}, \dots, v_{ma_m}$, one per clause group. For any two of these vertices v_{ja_j}, v_{ka_k} ($j \neq k$): both literals are true under the same assignment, so they cannot be complementary (a variable and its negation cannot both be true). Hence, by construction, an edge exists between them. Since this holds for every pair, the m chosen vertices form a clique of size m .
- $(\Leftarrow) G \text{ has a clique of size } m \Rightarrow \phi \text{ is satisfiable}$. Since no two vertices in the same clause-group are adjacent, an m -clique can contain *at most one* vertex from each of the m groups; as there are exactly m groups and the clique has m vertices, it contains *exactly one* vertex per

group. Because all pairs in the clique are adjacent, by construction no two chosen literals are complementary, so setting every chosen literal to **true** is a *consistent* partial assignment (no variable is asked to be both true and false); extend it arbitrarily to the remaining variables. Under this assignment, every clause C_j has its chosen literal true, so every clause – and hence ϕ – is satisfied.

Thus $\phi \in 3\text{-SAT} \iff G$ has a clique of size $k = m$, so this is a valid polynomial-time reduction, proving $3\text{-SAT} \leq_p \text{Clique}$.

Consequence. Since 3-SAT is NP-complete and $3\text{-SAT} \leq_p \text{Clique}$, Clique is **NP-hard**. Also, $\text{Clique} \in NP$ (a candidate set of k vertices is a certificate that can be checked for pairwise adjacency in $O(k^2)$ time). Hence **Clique is NP-complete**.

Step 2: $\text{Clique} \leq_p 3\text{-SAT}$

Since $\text{Clique} \in NP$ (shown above), the Cook–Levin theorem guarantees that *every* problem in NP – in particular Clique – reduces to 3-SAT in polynomial time: $\text{Clique} \leq_p 3\text{-SAT}$.

For completeness, here is also a direct, explicit reduction. Given $(G = (V, E), k)$:

- Observe that G has a clique of size k if and only if the complement graph $\bar{G} = (V, \bar{E})$, where $\bar{E} = \{(u, v) : u \neq v, (u, v) \notin E\}$, has an **independent set** of size k (a set with no edges of $\bar{G}$ among them, i.e. all pairs *are* edges of G).
- Independent Set reduces to 3-SAT: introduce a boolean variable x_v per vertex meaning “ v is chosen”; for every edge $(u, v) \in \bar{E}$, add the clause $(\neg x_u \vee \neg x_v)$ forbidding both endpoints from being chosen; encode the cardinality constraint “at least k of the x_v are true” using standard polynomial-size cardinality gadgets (each expressible using $O(1)$ auxiliary variables and 3-literal clauses per unit, following the same technique used to build any polynomial-size boolean circuit into 3-CNF, per Cook–Levin’s construction).

This composed reduction (complement graph + IS-to-3SAT encoding) runs in polynomial time, giving an explicit $\text{Clique} \leq_p 3\text{-SAT}$.

Conclusion

We have shown $3\text{-SAT} \leq_p \text{Clique}$ and $\text{Clique} \leq_p 3\text{-SAT}$, so the two problems are polynomial-time interreducible. Equivalently: both problems are NP-complete, and by the very definition of NP-completeness every NP-complete problem reduces to every other one in polynomial time. Therefore, **3-SAT and Clique are equally hard**. ■

Part B: TSP is NP-Complete (via reduction from Hamiltonian Cycle)

Problem statements

Hamiltonian Cycle (HC) – decision. Given a graph $G = (V, E)$, does G contain a Hamiltonian cycle, i.e. a cycle that visits every vertex exactly once and returns to the start? (HC is a classical NP-complete problem, itself provable NP-complete by reduction from 3-SAT or from Vertex Cover; we take this as known/given.)

Travelling Salesman Problem (TSP) – decision. Given a complete graph $G' = (V, E', w)$ with non-negative integer edge weights $w : E' \rightarrow \mathbb{Z}^{\geq 0}$, and an integer k , does G' have a Hamiltonian cycle (a "tour") of total weight $\leq k$?

Step 1: TSP $\in NP$

A certificate for a "yes" instance is a permutation π of the $n = |V|$ vertices, describing the order in which the tour visits them. The verifier:

1. Checks that π is a permutation of all n vertices, i.e. every vertex appears exactly once – $O(n)$ time (e.g. using a boolean "seen" array).
2. Sums the weights of the n edges $(\pi(1), \pi(2)), (\pi(2), \pi(3)), \dots, (\pi(n), \pi(1))$ – $O(n)$ time (weights are looked up directly since G' is complete).
3. Accepts iff the total is $\leq k$.

This verification runs in time $O(n)$, polynomial in the input size, so **TSP** $\in NP$.

Step 2: HC $\leq_p$ TSP (TSP is NP-hard)

Let $G = (V, E)$ be an arbitrary instance of Hamiltonian Cycle, with $n = |V|$.

Construction (polynomial time). Build a TSP instance $(G' = (V, E', w), k)$:

- G' is the *complete* graph on the same vertex set V (i.e. E' contains every pair of distinct vertices).
- Define edge weights:

$$w(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E \\ 2 & \text{if } (u, v) \notin E \end{cases} \quad \text{for all } u \neq v \in V.$$

- Set $k = n = |V|$.

Building G' and computing all weights takes $O(n^2)$ time, which is polynomial in the size of the HC instance.

Correctness of the reduction. We must show:

$$G \text{ has a Hamiltonian cycle} \iff G' \text{ has a tour of weight } \leq k = n.$$

($\Rightarrow$) Suppose G has a Hamiltonian cycle $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_n \rightarrow v_1$. Every consecutive pair (v_i, v_{i+1}) (indices mod n) is an edge of E (that's what makes it a Hamiltonian cycle of G). In G' , this same cyclic sequence is also a valid tour (since G' is complete, every such pair is an edge of G' too), and by construction every one of its n edges has weight exactly 1 (because each corresponds to an edge of E). Hence the total tour weight is exactly $n = k$, so G' has a tour of weight $\leq k$.

($\Leftarrow$) Suppose G' has a tour of weight $\leq k = n$. Any tour in G' uses exactly n edges, and each edge has weight either 1 or 2. If even a single edge of the tour had weight 2, the total weight would be at least $(n-1) \cdot 1 + 1 \cdot 2 = n+1 > k$, contradicting the assumption that the tour's weight is $\leq n$. Therefore *every* edge of this tour must have weight exactly 1. By construction, weight-1 edges are precisely the edges that exist in the original graph E . So this tour uses only edges of E , visits every vertex exactly once, and returns to the start – i.e. it is exactly a Hamiltonian cycle of G .

Hence $G \in \text{HC} \iff (G', k) \in \text{TSP}$, so this is a valid polynomial-time reduction, proving $\text{HC} \leq_p \text{TSP}$. Since HC is NP-complete (hence NP-hard) and HC reduces to TSP, **TSP is NP-hard**.

Step 3: Conclusion

From Step 1, $\text{TSP} \in NP$. From Step 2, TSP is NP-hard. By definition, a problem that is both in NP and is NP-hard is **NP-complete**. Therefore:

TSP is NP-complete.

